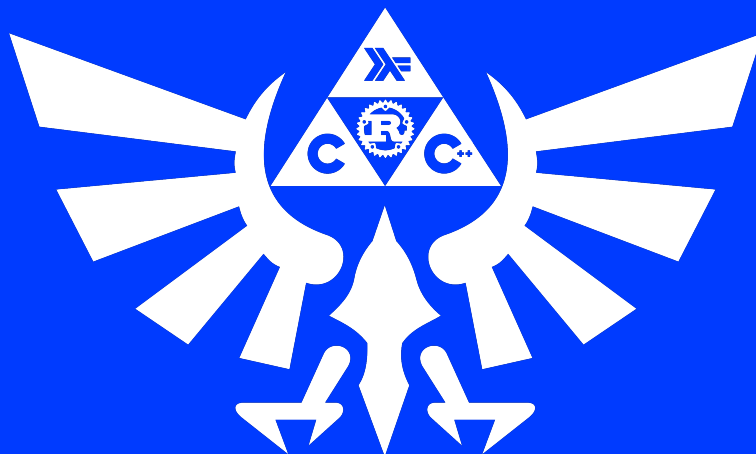


{EPITECH}

DAY 11

< WELCOME TO THE RUST UNIVERSE />



DAY 11



Project to deliver: Cargo.toml
language: Rust



- ✓ The totality of your source files, except all useless files (binary, temp files, obj files,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.

Today's goal is to discover a new programming language that will allow you to use concepts from functional programming, object oriented programming amongst other paradigms. Because not everything is black or white.

Congratulations, you won the job lottery ! You are to be assigned to the Space Facility of The Union in the team responsible of the Venus Space Program ! You're going to work on the first inhabited mission to Venus ! What a glorious job !

Introduction

Cargo - The Swiss Army knife

Cargo is the package manager, code formatter (`cargo fmt`), compilation tool (`cargo build`), unit test runner (`cargo test`) of rust (and many more). So clearly when I say "Swiss Army knife", I think I'm not exaggerating.



If rustup and cargo are not installed on your computer yet, you might want to do it now. Here is one of the ways you can do it : `curl https://sh.rustup.rs -sSf | sh`

Exercise 00 - The Beginning



Turn-in : Cargo project in `ex00` directory that could be compiled and run with `cargo run` (`cargo init` can be useful to initialize the project)

The Union wants you to spy on our neighbor, Mr C, and do the same thing as the first task that he did on January 6th (The "Z is for Zorglub" task).

Exercise 01 - Flying Sequence



Turn-in : `flying_sequence.rs` in a folder named `ex01`

The Union wants you to simulate the liftoff sequence from earth to orbit, before going to venus.



In this exercise you will find a lot of seconds to "wait", these are not real seconds. YOU DO NOT NEED to sleep the program for a real amount of seconds ! It's only a basic exercise about a new programming language. The timeout for the automated tests will be set to 5 seconds. DO NOT use functions like `thread::sleep()` for example.

You must write a function named

```
pub fn flying_sequence(mut countdown: u32, ignition: u32, secondary_activation: u32, thrust_duration: u32)
```



To test your work locally, you should initialize a cargo project (`cargo init`). You can define your function in a separate file (e.g., `flying_sequence.rs`) and use it in `main.rs` with `mod flying_sequence;` and `use flying_sequence::flying_sequence;`. Note that only the source file `flying_sequence.rs` is expected for the turn-in.

This function takes four parameters:

- `countdown` is the duration of the countdown before the liftoff
- `ignition` is the number of seconds before liftoff when the main engine is ignited
- `secondary_activation` is the number of seconds to wait after liftoff to fire the secondary engine. If this number is strictly lower than 30 seconds, the rocket will not be high enough, therefore you should [go back to the lab](#) !
- `thrust_duration` is the number of seconds before the secondary engine cuts off

For the whole duration of the flight you must inform The Union of the state of the rocket every 10 seconds.

In order to respect security standards, The Union wants you to cutoff the main engine 10 seconds before the secondary engine activation, then wait one second and decouple the main engine.



The Union expects you to be a professional, you should think about problems that may happen in the flying sequence. For example, ignition before the countdown begins is clearly a mistake and should not happen, in a case like this you should [go back to the lab](#) !

Here is a sample main function:

```
fn main() {  
    flying_sequence(5, 6, 30, 35);  
    println!("{}", "");  
    flying_sequence(5, 2, 30, 35);  
    println!("{}", "");  
    flying_sequence(10, 3, 159, 35);  
}
```

The output of the program should be the same as the one in the file `flying_sequence_output.txt` in the archive provided on the intranet along this subject.

Exercise 02 - Survivability



Turn-in : `survivability.rs` in a folder named `ex02`

Write a function with the following prototype

`pub fn compute_survivable_days(water_quantity: f32, water_cycles: u32, food_quantity: f32, nb_crew: u32) -> u32`
that computes the number of days that can be survived IN FULL with the amount of water and dehydrated food provided in the shuttle and with the number of astronauts given.

An astronaut needs 2 liters of water and 500g of food per day.

Here is a sample main function and its output:

```
fn main() {  
    println!("Survivable days : {}", compute_survivable_days(10.0, 2, 20.3, 5));  
    println!("Survivable days : {}", compute_survivable_days(14.0, 10, 56.3, 7));  
    println!("Survivable days : {}", compute_survivable_days(12.0, 26, 37.5, 3));  
}
```

```
Terminal
```

```
$> If the mission can embark 10L of water, 20.3kg of food and the water recycler can do 2  
water cycles... With 5 crew members, there is enough water for 2 day(s) and enough food for  
8 day(s).  
Survivable days : 2  
If the mission can embark 14L of water, 56.3kg of food and the water recycler can do 10  
water cycles... With 7 crew members, there is enough water for 10 day(s) and enough food  
for 16 day(s).  
Survivable days : 10  
If the mission can embark 12L of water, 37.5kg of food and the water recycler can do 26  
water cycles... With 3 crew members, there is enough water for 52 day(s) and enough food  
for 25 day(s).  
Survivable days : 25
```

Exercise 03 - Ownership



Turn-in : `ownership.rs` in a folder named `ex03`

The Union wants you to familiarize yourself with the following concepts : variable ownership, the move semantics, references, borrowing. These concepts will follow you everywhere as long as you use Rust so you better understand them correctly, otherwise The Union will be very disappointed and will end your career...

Here is a struct, a main function and its output. You must use this main function as is and create as much functions as you want to be able to compile it and get the corresponding output. You must not push your main function.

`ownership.rs` :

```
pub struct Chest {  
    pub helmet: u32,  
    pub suit: u32,  
}
```

`main.rs` :

```
fn main() {  
    let to_take: u32 = 2;  
    let mut equipment_chest: Chest = Chest { helmet: 10, suit: 23 };  
    let name1 = "Paul".to_string();  
    let name2 = "Laura".to_string();  
    let astronaut_1 = add_title(&name1);  
    let astronaut_2 = add_title(&name2);  
  
    println!("- Here is the contract of {}:\n{}", name1, sign_contract(&astronaut_1));  
    println!("- And here is the declaration of {}, ready to send to the press:\n{}\n",  
        astronaut_1, sign_press_declaration(&astronaut_1));  
    take_helmet(&mut equipment_chest, to_take);  
    take_suit(&mut equipment_chest, to_take);  
    println!("Number of Helmets: {} - Number of Suits: {}\n", equipment_chest.helmet,  
        equipment_chest.suit);  
  
    println!("- Here is the contract of {}:\n{}", name2, sign_contract(&astronaut_2));  
    println!("- And here is the declaration of {}, ready to send to the press:\n{}\n",  
        astronaut_2, sign_press_declaration(&astronaut_2));  
    take_helmet(&mut equipment_chest, to_take);  
    take_suit(&mut equipment_chest, to_take);  
    println!("Number of Helmets: {} - Number of Suits: {}", equipment_chest.helmet,  
        equipment_chest.suit);  
}
```

```
Terminal
$> cargo run | cat -e
- Here is the contract of Paul:$
I am honored to join the space program of The Union and I am willing to risk my life to
explore space. Therefore, I renounce every right I have to sue The Union in the event of my
death.$
Signed: Astronaut Paul$
- And here is the declaration of Astronaut Paul, ready to send to the press:$
I am thrilled that I have been chosen in the space program of The Union. I will make The
Union and my family proud by being a good astronaut, dedicated to the exploration of space.
Cheers and see you on Venus !$
Signed: Astronaut Paul$
$
Number of Helmets: 8 - Number of Suits: 21$
$
- Here is the contract of Laura:$
I am honored to join the space program of The Union and I am willing to risk my life to
explore space. Therefore, I renounce every right I have to sue The Union in the event of my
death.$
Signed: Astronaut Laura$
- And here is the declaration of Astronaut Laura, ready to send to the press:$
I am thrilled that I have been chosen in the space program of The Union. I will make The
Union and my family proud by being a good astronaut, dedicated to the exploration of space.
Cheers and see you on Venus !$
Signed: Astronaut Laura$
$
Number of Helmets: 6 - Number of Suits: 19$
```


Exercise 04 - Space Inventory



Turn-in : `space_inventory.rs` in a folder named `ex04`

The Union has a new task for you. The two astronauts for the Venus mission have a few training exercises to do with the tools (and maybe some weapons). Look at this main function below and do every function, struct and enum needed for this code to be able to work.

```
fn main() {
    let mut paul_tool: Item = Item {
        name: String::from("Space Hammer"), item_type: ItemType::Tool,
        price: 10.0, usage: 100,
    };
    let mut paul_weapon: Item = Item {
        name: String::from("Union Slasher"), item_type: ItemType::Weapon(125),
        price: 160.0, usage: 100,
    };
    let laura_tool: Item = Item {
        name: String::from("Sonic Screwdriver"), item_type: ItemType::Tool,
        price: 50.0, usage: 100,
    };
    let mut laura_weapon: Item = Item {
        name: String::from("Union Slicer"), item_type: ItemType::Weapon(212),
        price: 270.0, usage: 100,
    };
    println!(
        "Paul has a {} and it's {} new, he also has a {} doing a lot of damage : {} to be
        precise",
        paul_tool.name,
        paul_tool.usage,
        paul_weapon.name,
        get_damage(&paul_weapon)
    );
    println!(
        "Laura has a {} and it's {} new, she also has a big weapon {} doing a lot of
        damage : {}\n",
        laura_tool.name,
        laura_tool.usage,
        laura_weapon.name,
        get_damage(&laura_weapon)
    );
    let usage: u8 = 13;
    while !is_used(&paul_tool) {
        println!("Paul is using his tool and loses {}% of durability", usage);
        use_item(&mut paul_tool, usage);
        println!("Paul's tool has {} durability left and is worth {}", paul_tool.usage,
            paul_tool.price);
    }
    println!("\nIf we ever want to trade weapons with the locals once we're on Venus we
        better align our prices");
    harmonize_price(&mut paul_weapon, &mut laura_weapon);
    println!("Now the prices are : {} for Paul's weapon and {} for Laura's weapon", paul_
        weapon.price, laura_weapon.price);
}
```



The function `use_item` removes the provided percentage from `usage` (13% in this example). When `usage` falls below 25, the item's `price` is permanently reduced to 25% of its **original** value. Items of type `Tool` usually do 0 damage.

```
Terminal
$> Paul has a Space Hammer and it's 100 new, he also has a Union Slasher doing a lot of
damage : 125 to be precise
Laura has a Sonic Screwdriver and it's 100 new, she also has a big weapon Union Slicer
doing a lot of damage : 212
Paul is using his tool and loses 13% of durability
Paul's tool has 87 durability left and is worth 10
Paul is using his tool and loses 13% of durability
Paul's tool has 74 durability left and is worth 10
Paul is using his tool and loses 13% of durability
Paul's tool has 61 durability left and is worth 10
Paul is using his tool and loses 13% of durability
Paul's tool has 48 durability left and is worth 10
Paul is using his tool and loses 13% of durability
Paul's tool has 35 durability left and is worth 10
Paul is using his tool and loses 13% of durability
Paul's tool has 22 durability left and is worth 2.5
If we ever want to trade weapons with the locals once we're on Venus we better align our
prices
Now the prices are : 270 for Paul's weapon and 270 for Laura's weapon
```

Exercise 05 - Crew Manifest



Turn-in : `crew_manifest.rs` in a folder named `ex05`

Static arrays are for cowards who stay on Earth. In deep space, the mission parameters change every second. Crew members die, new synthetic androids are printed, and guests arrive.



You cannot rely on a fixed-size `[T; N]`. You need a structure that lives on the **Heap**. You need a collection that can breathe, grow, and shrink.

Instructions

Implement the following set of tools to manage your dynamic roster.

1. Allocation

```
pub fn create_crew(size: i32) -> Vec<String>
```

Initialize the manifest.

- ✓ We often know the estimated size of the crew beforehand. If `size` is positive, **optimize** the initialization to avoid useless reallocations during the first insertions.
- ✓ If `size` is useless (0 or negative), just give us an empty container.

2. Recruitment

```
pub fn join_crew(crew: &mut Vec<String>, name: String)
```

Add a new member to the end of the list.



Why does this function take `&mut`? What would happen to the ownership of the vector if we didn't pass a reference?

3. KIA (Killed In Action)

```
pub fn leave_crew(crew: &mut Vec<String>, name: &str)
```

Remove a member from the list.

- ✓ You must find them by their name.
- ✓ If multiple crew members share the same name (clones are common), remove only the **first** one encountered.
- ✓ If the name is not in the list, the function must do nothing and return peacefully.

4. Headcount

```
pub fn survey_crew(crew: &Vec<String>) -> usize
```

Return the current number of souls on board.



Why is the prototype using `&Vec` and not just `Vec`?

Example

```
fn main() {
    let mut crew = create_crew(10);

    join_crew(&mut crew, String::from("Ripley"));
    join_crew(&mut crew, String::from("Dallas"));
    join_crew(&mut crew, String::from("Brett"));

    println!("Current crew size: {}", survey_crew(&crew));

    println!("Removing Brett...");
    leave_crew(&mut crew, "Brett");

    println!("Removing Alien...");
    leave_crew(&mut crew, "Alien");

    println!("Final crew:");
    for member in crew {
        println!("- {}", member);
    }
}
```

```
Terminal
$> cargo run
Current crew size: 3
Removing Brett...
Removing Alien...
Final crew:
- Ripley
- Dallas
```

Exercise 06 - Gravity Well



Turn-in : Cargo project in `ex06` directory that could be compiled and run with `cargo run`

MAYDAY! MAYDAY!

Your spaceship is caught in the gravity well of a Neutron Star. The main fusion reactor is sputtering, and the gravity is pulling us in. The Captain is screaming orders, alarms are blaring, and you are the Chief Engineer.

You have a crate full of random debris scavenged from the cargo bay. Your only hope is to throw them into the reactor core one by one to generate enough energy to break orbit.

But be careful: The reactor is unstable. If it gets too hot, it melts down, and we all turn into stardust.

The Objective

You must create a program that simulates this emergency procedure. Your program will receive the initial state of the reactor and a list of debris via the command line.

You must simulate the reaction of each item in the order they are provided and determine the fate of the ship.

1. The Physics

The Reactor has two gauges: **Energy** and **Temperature**.

- ✓ **Target:** We need **2000 Energy** to break orbit.
- ✓ **Danger:** If Temperature exceeds **800**, the reactor melts down immediately.

You will be thrown different types of materials. Here is how they affect the reactor:

- ✓ **Uranium:** Standard nuclear fuel. Give 40 energy and 10 heat per kg.
- ✓ **Plutonium:** Highly unstable. Give 90 energy and 60 heat per kg.
- ✓ **OldBoot:** Organic carbon... mostly. Give 20 energy and 5 heat per kg.
- ✓ **Unknown:** Glowing alien rock. Give 20 heat per kg.

Special Rules for "Unknown":

The alien rock reacts to the current state of the reactor core *before* the rock enters it.

- ✓ If the current **Temperature** is an **even number** (0, 2, 4...), the rock resonates: **+400 Energy**.
- ✓ If the current **Temperature** is an **odd number** (1, 3, 5...), the rock dampens the reaction: **-150 Energy** (Energy can go negative).

2. The Interface

Your program must read arguments from the command line.

```
./gravity_well <Start_Energy> <Start_Temp> <Elem1> <Weight1> <Elem2> <Weight2> ...
```

- ✓ **Arg 1:** Initial Energy (Integer).
- ✓ **Arg 2:** Initial Temperature (Integer).
- ✓ **Arg 3+:** The list of debris, always in pairs: **Name** (String) followed by **Weight** (Integer).

Parsing:

- ✓ Element names are case-insensitive ("uranium", "Uranium", "URANIUM" are the same).
- ✓ If an element name is not recognized, print **Error: Unknown element** and stop the simulation.
- ✓ You can assume weights are always valid positive integers.

3. The Log

Your program is the flight recorder. It must output the status strictly as follows.

For every item processed, print:

Processing: <Element> (<Weight>kg)

Immediately after processing an item, print the new stats:

Reactor Stats: Energy <Current_E>, Temp <Current_T>

The simulation stops if:

1. **Meltdown:** Temperature strictly exceeds 800 (> 800). Print: `CRITICAL MELTDOWN!` and exit.
2. **Success:** Energy reaches or exceeds 2000 (≥ 2000). Print: `Orbit achieved!` and exit.
3. **Out of Fuel:** You processed all items, but Energy is still low. Print:
`Out of fuel. We drift into the void...`



Priority: The Meltdown check happens *after* the status update. If you reach 2000 Energy AND >800 Temp at the same time, the ship explodes before it can escape. **Meltdown takes priority over Success.**

Examples

```
Terminal
$> cargo run 0 0 uranium 50
Processing: Uranium (50kg)
Reactor Stats: Energy 2000, Temp 500
Orbit achieved!
```

```
Terminal
$> cargo run 0 700 plutonium 2
Processing: Plutonium (2kg)
Reactor Stats: Energy 180, Temp 820
CRITICAL MELTDOWN!
```

```
Terminal
$> cargo run 0 0 oldboot 1
Processing: OldBoot (1kg)
Reactor Stats: Energy 20, Temp 5
Out of fuel. We drift into the void...
```

v1

{EPITECH}